

Automatic Generation of Parallel Convolution Benchmarks for NoC-Based Manycore Architectures

Thomas S. Paula, Angelo Dal Zotto, Anderson Domingues Rodrigo C. Barros, César Marcon, Fernando Gehm Moraes
School of Technology, Pontifical Catholic University of Rio Grande do Sul – PUCRS – Porto Alegre, Brazil
thomas.paula@edu.pucrs.br, {angelo.dalzotto, rodrigo.barros, anderson.domingues, cesar.marcon, fernando.moraes}@pucrs.br

Abstract—This paper presents an automated code-generation pipeline for evaluating convolution workloads on NoC-based manycore systems, using Memphis-V as a case study. Starting from a compact workload specification, the proposed pipeline automatically generates sequential and parallel C applications, together with PyTorch reference outputs for correctness validation. The distributed version adopts input-channel parallelism: each convolution task processes one input channel and produces a partial result, which is then accumulated element-wise before ReLU, max pooling, and comparison against the reference output. This organization exposes the relationship between algorithmic parallelism, inter-task communication, and end-to-end execution time on a message-passing manycore. Experimental results on a 3x3 Memphis-V instance show that the parallel implementation maintains nearly constant execution time as the number of input channels increases, whereas the sequential baseline grows almost linearly. Across the evaluated design points, the parallel version achieves speedups from 2.3x to 4.1x. These results show that automatic code generation is an effective way to explore workload partitioning and communication tradeoffs on NoC-based manycores without rewriting the benchmark for each configuration.

Index Terms—NoC-based manycore, MEMPHIS-V, convolution benchmark, code generation, parallel embedded systems.

I. INTRODUCTION

In many vision workloads, convolutional layers dominate inference cost in both computation and data movement [1, 2]. As a result, they are a central concern in the design of efficient deployment strategies, particularly for edge systems with limited compute capability, memory capacity, and energy budget. NoC-based manycore architectures are well suited to this problem because they integrate distributed processing elements with scalable on-chip communication, providing a versatile platform for parallel convolution execution [3]–[5].

Although this architectural match is well-understood, practical evaluation of convolutional neural networks (CNNs) on manycore systems still faces an important methodological limitation. Existing proposals often rely on handcrafted benchmarks tailored to specific tensor shapes, layer configurations, or task-mapping strategies [6, 7]. As a result, evaluating different numbers of input and output channels or feature map dimensions usually requires manual code changes, reducing reproducibility and slowing design-space exploration. This limitation is even more relevant when the goal is not only to measure the end-to-end execution time but also to isolate the effects of computation, communication, synchronization, and workload partitioning in problem sizes [8, 9]. Thus, automated and parameterized benchmark generation remains necessary for systematic evaluation of CNNs on NoC-based manycores.

To address this limitation, we propose a Python-based code-generation pipeline¹ that automatically produces ready-to-run sequential and parallel convolution benchmarks in C for the open-source Memphis-V manycore [10]. The generated parallel version partitions convolution across input channels, exchanges partial results through message passing, and validates the final output against a PyTorch reference. By generating benchmark variants from a compact parameter set rather than manually rewriting each case, the proposed pipeline supports systematic exploration of tensor dimensions, workload partitioning, communication overhead, and end-to-end execution time. Rather than proposing a new convolution accelerator or dataflow, this work focuses on automating the generation of benchmark applications that make communication, reduction, and partitioning costs observable on a NoC-based manycore. The main contributions of this work are as follows:

- A Python-based, parameterized code-generation pipeline based on Jinja2 [11] templates that produces sequential and parallel C benchmarks for arbitrary combinations of input size, input channels, and output channels;
- An end-to-end correctness-validation methodology based on PyTorch reference outputs, enabling automatic verification of convolution, ReLU, and max pooling results;
- A systematic benchmark study with 36 experiments that quantifies the speedup and stage-level behavior of an input-channel-parallel convolution benchmark relative to sequential execution.

This paper is organized as follows. Section II reviews manycore architectures and the basics of CNNs. Section III describes the code-generation pipeline and the structure of the generated application. Section IV presents and discusses the experimental results. Section V concludes the paper.

II. BACKGROUND

A. NoC-based manycores and Memphis-V

NoC-based manycore architectures integrate tens to hundreds of Processing Elements (PEs) connected through a Network-on-Chip (NoC) [12, 13]. Manycore systems decouple computation from communication, providing scalability but also introducing latency and bandwidth constraints that must be considered when partitioning workloads [13].

This work adopts the open-source NoC-based manycore Memphis-V [10], built on Memphis [14], as a case study. It provides a clock-cycle-accurate hardware model, a complete software stack with an embedded Operating System (OS), and support for inter-task communication implemented through

¹Project repository: <https://github.com/gaph-pucrs/conv-benchmarks>

a message-passing API. Figure 1 details the Memphis-V manycore. Figure 1a shows an overview of the homogeneous region of Memphis-V with PEs and the external region with peripherals connected to the routers at the borders.

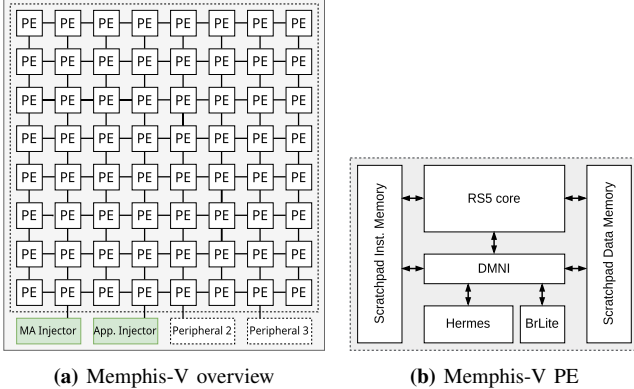


Fig. 1: Memphis-V MCSoc overview and PE organization.

Figure 1b details the PE. Its core is the RISC-V RS5 processor [15]. RS5 is connected to two dual-port scratchpad memories for instruction and data. Two NoCs are responsible for the communication: a packet-switching *Hermes* router [16], and a broadcast *BrLite* router [17]. In Memphis-V, Hermes has 32-bit flits, matching RS5's word size.

B. Convolution in Convolutional Neural Networks

A convolutional neural network (CNN) is a type of neural network that learns hierarchical feature representations from structured inputs such as images. A 2D convolutional layer transforms an input tensor $\mathcal{X} \in \mathbb{R}^{C_{in} \times H \times W}$ into a pre-activation output tensor $\mathcal{Z} \in \mathbb{R}^{C_{out} \times H' \times W'}$ through a learnable weight tensor $\mathcal{W} \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$ (omitting bias terms for clarity). Following modern convention, this operation is implemented as a cross-correlation:

$$\mathcal{Z}_{f,i,j} = \sum_{c=0}^{C_{in}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathcal{X}_{c, si+p, sj+q} \mathcal{W}_{f,c,p,q} \quad (1)$$

where s denotes the stride, f the output channel index, and (p, q) the spatial offsets within the $k \times k$ receptive field. For unit stride ($s = 1$) and no padding, the output dimensions are $H' = H - k + 1$ and $W' = W - k + 1$. Non-linearity is then introduced by the Rectified Linear Unit (ReLU), $\mathcal{A} = \sigma(\mathcal{Z})$, with $\sigma(x) = \max(0, x)$. In the architecture considered here, spatial downsampling is subsequently performed via max pooling using a 2×2 window with stride 2.

In this work, we employ a channel-wise parallelization strategy. Let $\{\mathcal{C}_m\}_{m=1}^M$ be a partition of the input-channel index set $\{0, \dots, C_{in}-1\}$. For each partition $\mathcal{C}_m$, we compute an independent partial sum tensor:

$$\mathcal{P}_{f,i,j}^{(m)} = \sum_{c \in \mathcal{C}_m} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathcal{X}_{c, si+p, sj+q} \mathcal{W}_{f,c,p,q} \quad (2)$$

The full convolution output is then recovered through the reduction:

$$\mathcal{Z}_{f,i,j} = \sum_{m=1}^M \mathcal{P}_{f,i,j}^{(m)} \quad (3)$$

ReLU and max pooling are applied only after this reduction. This dataflow exposes task-level parallelism but also introduces synchronization and reduction overheads absent in the sequential formulation. We adopt input-channel parallelism because it provides a simple one-task-per-channel decomposition that exposes both parallel computation and the cost of reduction and synchronization.

III. CODE GENERATION PIPELINE

The proposed pipeline automatically generates convolution benchmark applications from a compact set of user-defined parameters. Figure 2 presents the pipeline: (1) reference outputs generation via PyTorch, (2) configuration dictionary construction, and (3) application folder generation using Jinja2 templates.

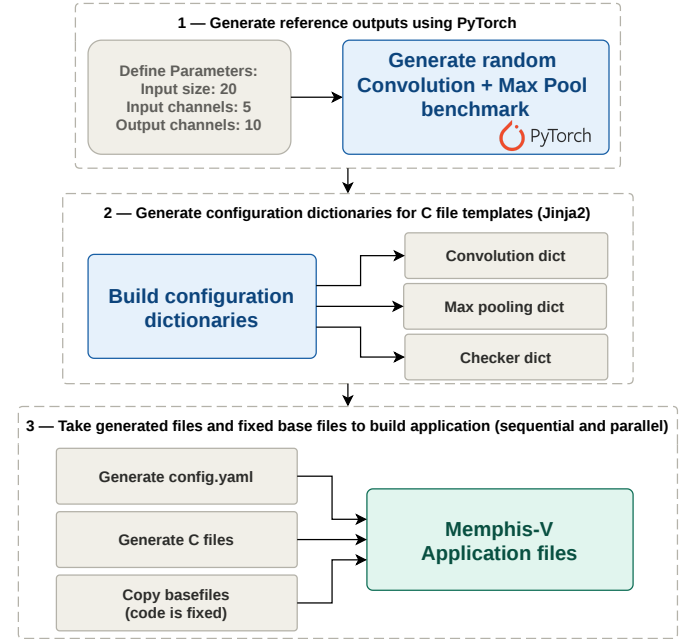


Fig. 2: Overview of the code-generation pipeline.

A. Reference Output Generation

The first stage uses PyTorch to generate reference tensors for convolution, ReLU, and max pooling. A convolution layer is instantiated with C_{in} input channels, C_{out} output channels, 3×3 kernels, no bias, stride 1, and no padding. Input tensors are initialized uniformly in the interval $[-10, 10]$ using a fixed random seed to ensure reproducibility. The pipeline then applies convolution, ReLU, and 2×2 max pooling in sequence, storing intermediate tensors produced at each stage. For an input tensor of size $C_{in} \times H \times W$, the final reference output has shape $C_{out} \times \lfloor (H-2)/2 \rfloor \times \lfloor (W-2)/2 \rfloor$.

B. Configuration Dictionary Construction

In the second stage, the generated tensors are flattened into one-dimensional integer arrays and combined with shape metadata to form Python configuration dictionaries. For the parallel implementation, three types of dictionaries are created, as illustrated in Figure 3: one dictionary per convolution task, containing a single input channel and the corresponding

kernel slice; one dictionary for the max pooling task, which accumulates partial tensors and performs post-processing; and one dictionary for the checker task, which stores the expected flattened output. The sequential implementation uses a single dictionary describing the full end-to-end computation.

```
{
  {
    "name": "convolution_0",
    "input_channels": 1,
    "input_height": 4,
    "input_width": 4,
    "output_channels": 3,
    "kernel": "{ 2, 6, 1, ..., 4}",
    "input": "{ -2, -7, -10, ..., -3}"
  },
  {
    "name": "convolution_1",
    "input_channels": 1,
    "input_height": 4,
    "input_width": 4,
    "output_channels": 3,
    "kernel": "{ -3, 9, -8, ..., 3}",
    "input": "{ 3, 7, 5, ..., 6}"
  }
}

{
  "name": "maxpool",
  "input_height": 2,
  "input_width": 2,
  "output_channels": 3,
  "convolution_tasks":
    "[convolution_0, convolution_1]"
}

{
  "name": "checker",
  "num_feature_maps": 3,
  "feature_map_height": 1,
  "feature_map_width": 1,
  "expected": "{240, 258, 263}"
}
```

(a) Convolution dictionary

(b) Max pooling dictionary

(c) Checker dictionary

Fig. 3: Example of dictionaries for a 4×4 input with 2 input channels and 3 output channels. Each convolution task (convolution_0, convolution_1) receives its input slice and kernel. The max pooling task lists its upstream dependencies, and the checker task holds the expected output for correctness verification.

C. Code Generation

In the final stage, each configuration dictionary is rendered through a Jinja2 template to produce C source files. The parallel application uses three template families – convolution.c.j2, maxpool.c.j2, and checker.c.j2 (Figure 4) – whereas the sequential application uses a single sequential.c.j2 template that encodes the entire convolution pipeline as one task. The generator script takes application_name, input_size, input_channels, and output_channels as inputs and produces sequential and parallel application folders, each containing the C sources, header files, Makefile, and config.yaml file required by Memphis-V.

D. Parallel Application Architecture

The parallel application decomposes the convolution into C_{in} independent tasks, each mapped to a separate PE. Each convolution task receives a single input channel and the corresponding kernel weights for all output channels, computes a partial convolution tensor, and transmits the result through the message-passing API. A downstream max pooling task waits for all partial tensors, accumulates them element-wise to reconstruct the full convolution result, applies ReLU, and then performs 2×2 pooling before forwarding the final tensor to the checker. The checker compares the received values against the PyTorch reference and reports pass/fail for each output channel. Figure 5 illustrates an example execution schedule.

E. Experimental Setup

All experiments run on Memphis-V, configured as a 3×3 mesh, assuming one task mapped per PE. The benchmark

```
// input dimensions
const int INPUT_CHANNELS = ({ input_channels });
const int INPUT_HEIGHT = ({ input_height });
const int INPUT_WIDTH = ({ input_width });

// convolution kernel
const int KERNEL_HEIGHT = 3;
const int KERNEL_WIDTH = 3;
const int KERNEL_STRIDE = 1;
const int OUTPUT_CHANNELS = ({ output_channels });

// convolution output
const int CONV_OUTPUT_HEIGHT = (INPUT_HEIGHT - KERNEL_HEIGHT) + 1;
const int CONV_OUTPUT_WIDTH = (INPUT_WIDTH - KERNEL_WIDTH) + 1;
const int CONV_OUTPUT_SIZE = OUTPUT_CHANNELS * CONV_OUTPUT_HEIGHT *
    CONV_OUTPUT_WIDTH;

const int NUM_FEATURE_MAPS = ({ num_feature_maps });
const int FEATURE_MAP_HEIGHT = ({ feature_map_height });
const int FEATURE_MAP_WIDTH = ({ feature_map_width });
const int MAXPOOL_TASK_ID = maxpool;

const int NUM_INPUT_VALUES = FEATURE_MAP_HEIGHT * FEATURE_MAP_WIDTH;
const int MSG_SIZE = NUM_INPUT_VALUES * sizeof(int);

/**
 * @brief Perform a comparison between output of max pool and a reference vector
 *        for each feature map.
 *
 * @param expected Flattened array of expected values.
 */
void perform_check(int *expected)
```

Fig. 4: Excerpts from Jinja2 templates. *Top:* sequential.c.j2 defines layer dimensions and kernel parameters as C constants injected at generation time. *Bottom:* checker.c.j2 injects the expected output array and compares it against the result received from the max-pooling task.

sweeps three hyperparameters: input_size $\in \{32, 64, 128\}$, $C_{in} \in \{3, 4, 5\}$, and $C_{out} \in \{4, 8\}$, yielding $3 \times 3 \times 2 = 18$ parameter combinations, each executed in both sequential and parallel modes for a total of 36 experiments. Memory size is scaled with problem size; the largest experiments (input size 128, $C_{out} = 8$) require up to 2048 KB for the sequential task and 512 KB per PE for the parallel tasks. Execution logs are parsed by a post-processing script that extracts per-task start and end timestamps, compiled into a CSV, and analyzed in a Jupyter [18] notebook. The manycore is simulated at the RTL level, with clock-cycle accuracy, at 100 MHz.

IV. RESULTS

A. Sequential vs. Parallel Execution Time

Table I reports the average total execution time for all evaluated configurations. For a fixed input size and number of output channels, the parallel implementation varies only slightly as C_{in} increases. This behavior is consistent with the adopted input-channel partitioning: increasing C_{in} creates additional convolution tasks but does not significantly increase the work per task. As a result, end-to-end latency is dominated by the slowest convolution task together with communication, accumulation, and post-processing overheads. In contrast, the sequential baseline grows approximately linearly with C_{in} .

Across all evaluated configurations, the parallel implementation achieves speedups ranging from $2.3\times$ to $4.13\times$, with larger gains observed for larger inputs and higher numbers of output channels (see Table I). For example, with input size 128, $C_{out} = 8$, and $C_{in} = 5$, the sequential execution time is 1326.7ms, whereas the parallel execution time is 321.5ms, corresponding to a speedup of $4.13\times$.

B. Per-Stage Analysis

Convolution Stage. Table II reports the mean per-convolution task time across all parallel experiments. Because each task

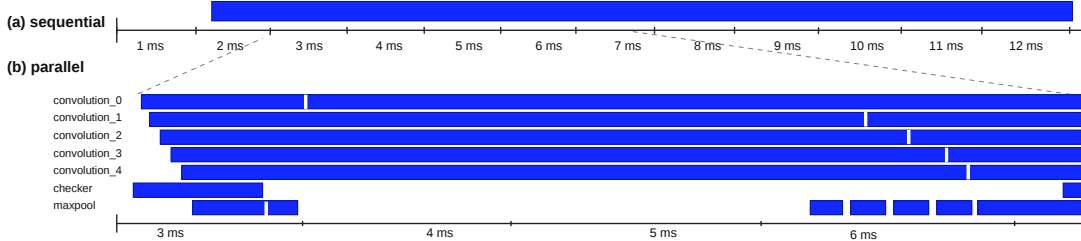


Fig. 5: Scheduling views for sequential and parallel execution of the same benchmark. The sequential run uses a single PE over the entire execution window. The parallel run assigns one PE per input channel, one PE for accumulation, ReLU, max pooling (*maxpool* task), and one PE for the checker task. The schedule highlights both task-level overlap and the serialization introduced by downstream processing.

TABLE I: Average execution time (ms) and speedup ($\times$) for the 36 convolution setups, varying the number of input (C_{in}) and output (C_{out}) channels, for different input sizes.

Input size	Input channels	4 Output channels			8 Output channels		
		Sequent.	Parallel	$\times$	Sequent.	Parallel	$\times$
32	3	23.5	10.2	2.30	45.7	18.3	2.50
	4	30.8	10.3	2.99	59.8	18.5	3.23
	5	37.8	10.5	3.60	73.2	18.6	3.94
64	3	104.1	41.2	2.53	200.2	77.4	2.59
	4	127.7	41.7	3.06	251.2	78.0	3.22
	5	157.0	42.2	3.72	308.5	78.5	3.93
128	3	409.8	168.2	2.44	788.3	317.6	2.48
	4	554.3	170.2	3.26	1023.2	319.6	3.20
	5	653.1	172.1	3.80	1326.7	321.5	4.13

processes one input channel, its local workload remains nearly constant as C_{in} varies. This explains why, for a fixed input size, the measured per-task time is similar across all C_{in} values. Increasing the input size from 32 to 128 raises convolution time by about $16.8\times$ with $C_{out} = 4$ (9.3 ms to 156 ms) and $17.7\times$ with $C_{out} = 8$ (17.3 ms to 305.4 ms), consistent with the increase in output pixels per feature map. For a fixed kernel size, the cost of each partial convolution scales approximately with $H_{out}W_{out}C_{out}$.

TABLE II: Average convolution task execution time.

Input size	Input channels	4 Output channels (ms)	8 Output channels (ms)
32 $\times$ 32	3,4,5	9.3	17.3
64 $\times$ 64	3,4,5	38.0	74.3
128 $\times$ 128	3,4,5	156.0	305.4

Max Pooling Stage. The max pooling task processes all C_{out} feature maps sequentially after receiving them from the convolution PEs. This stage also reflects the cost of waiting for the upstream partial results to be assembled before post-processing can be completed. Its execution time (Table III) scales with both the input size and C_{out} : doubling C_{out} roughly doubles the pooling time, since twice as many feature maps must be received, processed, and forwarded to the checker.

Checker Stage. The checker task (Table IV) waits for the max pooling output to arrive and terminates after the last pooled channel is received, performing correctness verification against the PyTorch reference. Its own computational cost is negligible compared with the rest of the pipeline. Therefore, the measured checker duration is best interpreted not as the isolated cost of checking, but as an end-to-end latency indicator observed at the sink of the parallel application.

TABLE III: Max pooling task execution time (ms). Execution time increases with input size and with C_{out} , as both affect the volume of data processed by the single pooling PE.

Input size	Input channels	4 Output channels (ms)	8 Output channels (ms)
32 $\times$ 32	3	9.9	18.1
	4	10.1	18.2
	5	10.2	18.3
64 $\times$ 64	3	40.8	77.1
	4	41.3	77.6
	5	41.8	78.1
128 $\times$ 128	3	157.5	316.9
	4	169.4	318.8
	5	171.3	320.7

TABLE IV: End-to-end latency observed at the checker task.

Input size	Input channels	4 Output channels (ms)	8 Output channels (ms)
32 $\times$ 32	3	10.2	18.3
	4	10.3	18.5
	5	10.5	18.6
64 $\times$ 64	3	41.2	77.4
	4	41.7	78.0
	5	42.2	78.5
128 $\times$ 128	3	168.2	317.6
	4	170.2	319.6
	5	172.1	321.5

V. CONCLUSION

This paper presented a code-generation pipeline that automates the creation of sequential and parallel convolution benchmarks for NoC-based manycores. By combining PyTorch-based reference generation with Jinja2 templates, the proposed pipeline reduces the manual effort required to create, validate, and execute benchmark instances across multiple configurations. Experimental results show that the generated input-channel-parallel implementation consistently outperforms the sequential baseline, achieving speedups from $2.30\times$ to $4.13\times$. The stage-level analysis further shows that per-task convolution time remains nearly constant with respect to C_{in} , while downstream stages concentrate the overhead associated with reduction and post-processing. Overall, the results demonstrate that automated benchmark generation is an effective method for studying the trade-offs between parallel computation and communication overhead in NoC-based manycore systems.

Future work includes distributing max pooling across multiple PEs to reduce serialization overhead, extending the pipeline to additional layer types such as fully connected layers and batch normalization, and automating memory-footprint estimation to eliminate manual memory-size tuning.

ACKNOWLEDGMENTS

This work was financed in part by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), Finance Code 001; Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), 308182/2023-5, 305621/2024-6; Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS), 23/2551-0002200-1.

REFERENCES

- [1] R. Raj and A. Kos, "An Extensive Study of Convolutional Neural Networks: Applications in Computer Vision for Improved Robotics Perceptions," *Sensors*, vol. 25, no. 4, pp. 1–19, 2025, <https://doi.org/10.3390/s25041033>.
- [2] G. Rangel, J. C. Cuevas-Tello, J. Nunez-Varela, C. Puente, and A. G. Silva-Trujillo, "A Survey on Convolutional Neural Networks and their Performance Limitations in Image Recognition Tasks," *Journal of Sensors*, 2024, <https://doi.org/10.1155/2024/2797320>.
- [3] R. Cordova-Cardenas, D. Amor, and Álvaro Gutiérrez, "Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems," *Electronics*, vol. 14, no. 24, pp. 1–39, 2025, <https://doi.org/10.3390/electronics14244877>.
- [4] W. Zhu, Y. Chen, and Z. Lu, "Pooling On-the-Go for NoC-Based Convolutional Neural Network Accelerator," in *Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, ser. Lecture Notes in Computer Science, 2025, pp. 109–118, https://doi.org/10.1007/978-3-031-78380-7_9.
- [5] W. A. Nunes, A. V. C. D. Santos, C. A. M. Marcon, and F. G. Moraes, "Accelerating Machine Learning using RISC-V Vector Extension in a Manycore Platform," in *VLSI-SoC*, 2025, pp. 1–5, <https://doi.org/10.1109/VLSI-SoC64688.2025.11421728>.
- [6] F. Lesniak, A. Gutermann, T. Harbaum, and J. Becker, "Enhanced Accelerator Design for Efficient CNN Processing with Improved Row-Stationary Dataflow," in *GLSVLSI*, 2024, pp. 151–157, <https://doi.org/10.1145/3649476.3658737>.
- [7] S. P. Nallabelli and S. Sampat, "Enhanced Convolutional Neural Network Accelerators with Memory Optimization for Routing Applications," *PLOS ONE*, vol. 20, no. 4, p. e0320813, 2025, <https://doi.org/10.1371/journal.pone.0320813>.
- [8] G. Naran, C. Ogbogu, J. R. Dopp, and P. P. Pande, "TEFLON: Thermally Efficient Dataflow-aware 3D NoC for Accelerating CNN Inferencing on Manycore PIM Architectures," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 5, pp. 78:1–78:23, 2024, <https://doi.org/10.1145/3665279>.
- [9] A. Dehghani, "An Efficient NoC-Based Architecture for Convolutional Neural Networks on Multicore Systems," *The Journal of Supercomputing*, vol. 81, no. 9, pp. 1–24, 2025, <https://doi.org/10.1007/s11227-025-07526-x>.
- [10] Memphis-V, "Memphis-V," 2026, <https://github.com/gaph-pucrs/Memphis-V>.
- [11] A. Ronacher and the Pallets Project, "Jinja: A fast, expressive, extensible templating engine," 2008. [Online]. Available: <https://jinja.palletsprojects.com/>
- [12] S. Biglari, F. Hosseini, A. Upadhyay, and H. Zhao, "Survey of Network-on-Chip (NoC) for Heterogeneous Multicore Systems," in *MCSoc*, 2024, pp. 155–162, <https://doi.org/10.1109/MCSoc64144.2024.00036>.
- [13] P. C. D. Paris and E. C. Pedrino, "A high-level simulator for network-on-chip," *Integrated Computer-Aided Engineering*, vol. 32, no. 1, pp. 55–71, 2025, <https://doi.org/10.3233/ICA-240743>.
- [14] M. Ruaro, L. L. Caimi, V. Fochi, and F. G. Moraes, "Memphis: a framework for heterogeneous many-core SoCs generation and validation," *Springer Design Automation for Embedded Systems*, vol. 23, no. 3–4, pp. 103–122, 2019, <https://doi.org/10.1007/s10617-019-09223-4>.
- [15] W. A. Nunes, A. E. Dal Zotto, C. d. S. Borges, and F. G. Moraes, "RS5: An Integrated Hardware and Software Ecosystem for RISC-V Embedded Systems," in *LASCAS*, 2024, pp. 1–5, <https://doi.org/10.1109/LASCAS60203.2024.10506171>.
- [16] F. G. Moraes, N. L. V. Calazans, A. V. Mello, L. H. Möller, and L. C. Ost, "HERMES: an Infrastructure for Low Area Overhead Packet-switching Networks on Chip," *ACM Integration, the VLSI Journal*, vol. 38, no. 1, pp. 69–93, 2004, <https://doi.org/10.1016/j.vlsi.2004.03.003>.
- [17] E. Wachter, L. L. Caimi, V. Fochi, D. Munhoz, and F. G. Moraes, "BrNoC: A broadcast NoC for control messages in many-core systems," *Elsevier Microelectronics Journal*, vol. 68, pp. 69–77, 2017, <https://doi.org/10.1016/j.mejo.2017.08.010>.
- [18] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds. IOS Press, 2016, pp. 87 – 90, <https://doi.org/10.3233/978-1-61499-649-1-87>.